

RYO-CHAN Hackathon 2026 — Project Submission Form

Filled against the organiser's own template, downloaded from `https://ryobuild.com/RYOCHAN-Hackthon-Project-Submission-Form.pdf` on 2026-09-07 and committed unchanged beside this file as `RYOCHAN-Hackathon-Project-Submission-Form-BLANK.pdf`. That PDF has no interactive fields, so this is the same form filled in, in the same order, and rendered to `project-submission-form.pdf` by `scripts/submission_pdf.py`.

Members

Total Members		1
Name	Role	Email
Pugar Huda Mantoro	Sole builder: architecture, backend, agents, interface, tests	hudapugar@gmail.com

Discord: Lynx (hajislamet) · GitHub: PugarHuda

Overview

Field	Value
Project	Nota
Track	Track 1 (Autonomous Agents), Track 2 (Dashboards & Interfaces) and Track 3 (New Skills). Each is judged independently.
Problem	An AI opinion about a market is unfalsifiable. You cannot tell whether the number in a sentence came from the evidence or from the model, whether a missing feed was quietly treated as zero, or whether the answer would be the same tomorrow. Nothing in a chat log can be re-run, so nothing in it can be audited, and a confident paragraph is indistinguishable from a careful one.
Solution	Nota turns each decision into a receipt that reproduces. Three specialised agents argue over live RYO evidence and cite dotted paths into it; a judge weighs them by each agent's own Brier score from past calls; risk sizing is pure ATR arithmetic with no model in it. <code>nota replay <id></code> rebuilds the decision from the stored evidence and the model output cached beside it, keyed by the model that produced them, and prints <code>identical: True</code> . That runs with no API key and no network, so anyone who clones the repository can check the claim. Nota also audits RYO itself: it recomputes RYO's RSI and ATR from public candles with Wilder's method, checks RYO's price against three exchanges, and compares its own sizing against RYO's published trade plan. When the sources disagree, the judge refuses to size a trade at all. And it keeps a record RYO does not: the RYO Verdict Scorecard locks every <code>deep_analysis</code> verdict and trade plan for 25 majors once a day and settles each plan on OKX's hourly candles (stop or +1R first, within 24 h and 72 h), so a reader can see whether RYO's verdict changes how often RYO's own plan works.
Key Features	Council of three agents with dotted-path citations, and citations pointing at absent evidence are dropped in code rather than trusted. Judge weighted by historical Brier score. Pure-ATR sizing that blocks rather than guesses. Replayable receipts, verifiable from the browser or the CLI. A derivatives gate: <code>positioning_check</code> rules each field of RYO's derivatives block citable or not without guessing RYO's undocumented definitions (the same value on two other tokens that day is <code>not_token_specific</code> ; on 18 September one 24 h open-interest change repeated across ETH, SOL, WIF and ONDO), and a withheld field is masked in the prompt and cannot be cited. The RYO Verdict Scorecard at <code>/scorecard</code> : daily locks, independent settlement, a within-day comparison of confluence states with a 90% interval bootstrapped over days, and a coverage count where every failed lock is a line; on day 0, 7 of 25 plans were long under a <code>cautious</code> verdict. Every agent

Field	Value
	is scored against the token's own base rate (how often it closed higher seven days later on same-volatility days, counted before the decision), not only against a coin flip. Diff-first dashboard that ranks what changed against the previous receipt by impact. Seven research skills in RYO's own envelope, served both on RYO's <code>/api/skills</code> paths and as a real MCP server at <code>POST /mcp</code> with tools and resources. <code>nota watch</code> autonomous loop with Telegram and Discord publishing. Public backing on receipts, scored when the call resolves. Open Graph receipt cards, markdown and JSON exports, <code>llms.txt</code> , keyboard-first accessible interface in light and dark. The landing page states no number of its own: it draws its marks, its verify target and its failure panel from <code>/api/decisions</code> , and a test asserts there is no evidence hash written into the page at all. That failure panel shows the most recent receipt made on incomplete evidence, with that receipt's own availability rows and warnings — currently 10 September, when the builder key began answering <code>UNAUTHENTICATED</code> and all five RYO sections failed at once while the three independent skills kept answering.
Target Users	Traders and analysts who need to justify a call rather than only make one; RYO itself, which can mount the seven skills as routes or call them over MCP; and hackathon judges, who can verify every claim on this page from outside the project.
Scope	Read-only research over RYO's six builder tools plus independent public sources. Practice trades only: sizes, stops and targets are recorded and marked against later prices, and no order is ever placed, signed or routed. Ships with a ledger snapshot that starts at four receipts made on live RYO evidence on 2026-09-07 and grows: a daily GitHub Actions cycle scores every decision that reached its seven-day horizon and commits the snapshot back, which is what turns the judge's Brier weighting from a design into a measurement. The same cycle locks the scorecard's 25 plans and settles the ones whose window has closed; that needs the builder key for the lock and nothing for the settlement. Scoring needs no key of any kind - the price comes from the exchange median - so the ledger kept calibrating after the builder key issued for this hackathon began answering 401 on 2026-09-10.
Limitations	Stated plainly rather than hidden. A council decision needs one LLM key; live RYO evidence needs the builder key; verification and the skills need neither. X's public syndication endpoint throttles hard and has tightened: it hit data-centre addresses first, and as of 2026-09-10 an ordinary connection gets <code>syndication HTTP 429</code> as well, so <code>x:</code> voices are best effort while <code>tg:</code> and <code>bs:</code> answer from both. <code>TAVILY_API_KEY</code> turns that into a search-backed fallback the envelope labels as undated; unset, the warning names the missing key rather than the voice going quietly silent. Brier weights only move once calls reach their seven-day horizon; with five scored by 18 September they already have (macro 1.0, technician 0.754, narrative 0.723), but five is far too few to trust, and the dashboard says so beside every such table. The hosted deployment reads its ledger from a snapshot because a serverless filesystem cannot be written; backings are kept in Postgres (Neon, free tier) so the public backing feature works there, while every other write still belongs to the CLI. With no <code>DATABASE_URL</code> at all — a plain clone — backings go to the ledger's own table, so nothing needs a database to run or to test. The receipt diff ranks changes with a documented heuristic, not a model. The scorecard began on 18 September, so at judging it holds about two weeks of lock days in what has so far been one market regime; its comparison between confluence states will most likely still read "not yet distinguishable", and the page says so instead of drawing an interval from too few days. Settlement uses hourly candles, so a stop and a target touched inside the same hour are recorded as <code>ambiguous</code> rather than ordered by guess, and the bracket is re-anchored to OKX's price because RYO and OKX are two feeds (a lock more than 2% apart is kept and never settled).

Tech Stack

Field	Value
Frontend Stack	No framework and no build step: four hand-written pages (the landing in English and Japanese, the receipt dashboard, the scorecard, and the walkthrough) of semantic HTML with native CSS

Field	Value
	and vanilla JavaScript, served by the same FastAPI process. One palette expressed with CSS <code>light-dark()</code> so light, dark and system all work from one definition, <code>prefers-reduced-motion</code> honoured, keyboard navigation and visible focus throughout. The hero mark is an inline SVG generated from the receipt's own evidence hash, and <code>/demo</code> pairs the walkthrough with a clickable transcript built from the recorder's own timings.
Backend Stack	Python 3.12, FastAPI and uvicorn, Pydantic v2 for every boundary type, httpx for outbound calls, SQLite in WAL mode as the ledger (evidence packs, LLM output cache, decisions, outcomes, and backings when no Postgres is configured), psycopg3 against Neon Postgres for backings on the hosted deployment, Typer for the <code>nota</code> CLI, Pillow for receipt cards, defusedxml for RSS, vaderSentiment for lexicon scoring. Deployed on Vercel as a framework-detected FastAPI app.
AI Model(s)	Provider-agnostic by design. Two adapters: Anthropic, and any OpenAI-compatible endpoint. Every shipped receipt was produced through Venice with <code>qwen3-235b-a22b-instruct-2507</code> . Replay is keyed by the model that produced a receipt rather than by whatever is configured now, which is what let an earlier receipt made under <code>anthropic/claude-haiku-4.5</code> keep verifying after the provider changed. Model outputs are cached by <code>(evidence hash, role, prompt version, model)</code> .
Other Technologies	Model Context Protocol in both directions: Nota is an MCP client of RYO (JSON-RPC <code>tools/call</code> , selectable with <code>RYO_TRANSPORT=mcp</code>) and an MCP server of its own over Streamable HTTP at <code>POST /mcp</code> , negotiating protocol versions 2026-07-28 through 2024-11-05 and serving both <code>tools/*</code> and <code>resources/*</code> . Public data sources, all keyless: OKX public market and derivatives data (daily and hourly candles, perp premium, open interest in coins, long/short account ratio), Hyperliquid's public info endpoint, CoinGecko, Coinbase, Kraken, alternative.me Fear & Greed, t.me/s channel previews, the Bluesky public AppView, X's public syndication endpoint, and CoinDesk, Cointelegraph, The Block and Decrypt RSS. Tavily or Venice web search when a key is present. Playwright drives the browser QA suite and records the walkthrough, <code>edge-tts</code> (GPL-3.0) reads the narration with a Microsoft neural voice and no key, and Remotion (Remotion License, free at this team size) composes the two; none of the three ships any code into the application.

Repository / Demo

Field	Value
Github Repository	https://github.com/RYO-Digital/ryochan-hackathon_repository-235 (the repository issued for this project; all final code is on <code>main</code>). Public mirror of the same history: https://github.com/PugarHuda/nota — listed in the official MCP registry as <code>io.github.PugarHuda/nota</code> , published 2026-09-10 from the repository over GitHub OIDC, so any MCP client can discover this server without being told its URL: https://registry.modelcontextprotocol.io/v0.1/servers?search=io.github.PugarHuda/nota
Demo Video	https://nota-ryo.vercel.app/demo — a narrated 3:04 walkthrough with a clickable transcript, served by the project itself, public and with no password; the bare file is at https://nota-ryo.vercel.app/demo.mp4 and is committed at <code>nota/static/demo.mp4</code> . It was recorded against the running app: a cursor travels to whatever the voice is describing and a box frames it, the verify button is pressed on camera, and every figure spoken is one this deployment produces on demand.
Documentation	<code>README.md</code> in the repository is the primary document: how a decision is made, the honesty rules the code enforces, the MCP server, the dashboard, the skills, and how to evaluate everything with zero keys. <code>docs/skills/SKILL-SPEC.md</code> is the skill contract; <code>docs/HACKATHON-ANALYSIS.md</code> records what was verified about the platform; <code>docs/SUBMISSION.md</code> tracks this submission. Live: https://nota-ryo.vercel.app and https://nota-ryo.vercel.app/lms.txt

Testing Information

Field	Value
How to run the Project	Clone the repository, <code>uv sync</code> , then <code>NOTA_DB=data/demo.db uv run nota serve</code> and open <code>http://127.0.0.1:8000</code> for the overview or <code>http://127.0.0.1:8000/app</code> for the dashboard itself. That serves the shipped ledger of receipts made on live RYO evidence, so the dashboard, the replay verification, the receipt cards, the exports and the skill runner all work before any key is set.
Prerequisites	Python 3.12 or newer and uv . No database server, no Docker, and no Node to run or evaluate the project. For the browser tests only: <code>uv run playwright install chromium</code> . Node is needed only to re-render the walkthrough video, which ships built.
Installation Steps	1. <code>git clone <repository> && cd nota</code> 2. <code>uv sync</code> 3. <code>cp .env.example .env</code> 4. optionally fill in the keys below 5. <code>uv run nota health</code> to confirm RYO is reachable.
Environment Variables	All optional for evaluation; <code>.env.example</code> lists every one with comments and no values. <code>RYO_MCP_KEY</code> for live RYO evidence, with <code>RYO_MCP_URL</code> and <code>RYO_TRANSPORT</code> (<code>rest</code> or <code>mcp</code>). One LLM key for the council: <code>ANTHROPIC_API_KEY</code> , or <code>NOTA_LLM=openai</code> with <code>OPENAI_BASE_URL</code> and <code>OPENAI_API_KEY</code> for any OpenAI-compatible provider, plus <code>NOTA_MODEL</code> and <code>NOTA_MAX_TOKENS</code> . <code>NOTA_DB</code> selects the ledger, <code>NOTA_READONLY=1</code> opens it immutably, <code>NOTA_PUBLIC_URL</code> sets the base for permalinks. Optional: <code>NOTA_VOICES</code> , <code>TAVILY_API_KEY</code> , <code>TELEGRAM_BOT_TOKEN</code> , <code>TELEGRAM_CHAT_ID</code> , <code>DISCORD_WEBHOOK_URL</code> . No real key is committed anywhere, and the git history was scanned to prove it.
Build Command	None. There is no build step: no bundler, no transpiler, no generated assets. <code>uv sync</code> installs the locked dependencies from <code>uv.lock</code> and the app runs from source.
Run Command	<code>uv run nota serve</code> for the interface and the API, or <code>uv run nota decide SOL</code> for one decision, <code>uv run nota watch SOL,BTC --every 3600 --notify</code> for the autonomous loop, <code>uv run nota replay <id></code> to verify a receipt, <code>uv run nota skill run price_crosscheck '{"symbol":"SOL"}'</code> for a skill. <code>uv run nota lock</code> and <code>uv run nota settle</code> for the scorecard. <code>uv run nota --help</code> lists all fourteen commands plus the <code>skill</code> group (<code>skill spec</code> , <code>skill run</code>).
Test Command	<code>uv run pytest -q</code> — 227 tests, currently all passing. It needs no network and no keys: unit tests, API tests, MCP server and transport tests, and seventeen browser QA checks that drive a real unicorn with Playwright over the shipped ledger, failing on any console error or failed request — including one that plays the shipped demo video and refuses a transcript chapter that starts past its end. Without Chromium installed the browser checks skip with a clear reason rather than failing.
Test Account(s)	None needed. Nothing in Nota has a login: there are no accounts, no sessions and no wallets. Backing a receipt asks only for a handle, which is a label rather than an identity, and the hosted deployment is read-only so it refuses backing with a 503 that explains why.

Declarations

- **No secret has ever been committed**, and the organiser's review of git history was anticipated. Re-checked on 2026-09-19 across all 91 commits: the exact values of the builder key and the LLM key in use were searched for in every patch and found in none. The first scan, at 49 commits and 448 blobs, ran against twelve credential patterns (RYO builder keys, Anthropic, OpenAI, OpenRouter, Venice, Tavily, Telegram bot tokens, Discord webhooks, AWS, GitHub and Vercel tokens, PEM private keys). The single match is `ryo_mcp_your_private_key`, the placeholder inside

RYO's own builder guide that this repository quotes at `docs/MCP-Builder-Guide.md`. No `.env` or credential file was added in any commit; `.env.example` carries names only.

- **No fabricated data is presented as real.** Every receipt prints its own source label. `live` means RYO answered; `recorded` means a real RYO response captured with `nota record` and replayed; `fixture` means a hand-built test fixture. `data_mode` is carried through from RYO per section, a value that could not be fetched stays `null` and is never turned into zero, and the risk layer refuses to size a trade on evidence RYO marks `simulated`.
- **Third-party code and resources are disclosed** in the README. No starter template was used; the repository began empty and every line of application code was written during the hackathon. Libraries: `httpx`, `pydantic`, `fastapi`, `uvicorn`, `typer`, `python-dotenv`, `anthropic`, `vaderSentiment`, `defusedxml`, `pillow`; development only: `pytest`, `respx`, `playwright`. The `taste-skill` design ruleset was read while reshaping the interface and is gitignored rather than vendored, so no file of it ships here.
- **Read-only.** The RYO surface used is read-only, and Nota never places, signs or routes an order. Practice positions exist only in the local ledger and are labelled as practice everywhere they appear.